

🔴 상태: 보류 (2026-08-19 판단) — 현재 기준은 STATUS.MD

Week 1(배선)은 사실상 이미 되어 있었다 — 브레이커 방깅은 `BulletTrail_Breaker` 프리팹 onHit, 버프·가드는 각 Ability asset에 연결됨(2026-07-14). Week 4(GC 측정)는 **축이 맞지 않아 철회** — 이 시스템의 명제는 "발동 3곳→1곳 · 스킬=조합 · 새 스킬 코드 0줄"로 **변경 비용**에 관한 것이라 캐스트당 힙 할당은 그것을 검증하지 않는다. 더구나 아래 가설은 **사실이 아니다**: `AbilityData.BuildRuntimeEffects()` 가 캐스트마다 `new List<IEffect>`를 판다. 측정하면 실제 병목도 아닌 곳을 포폴 숫자 때문에 최적화하게 된다. Week 2~3(가시화·GIF)은 연출 작업이라 우선순위가 낮다고 판단해 보류 — 되살릴 경우 이 문서를 그대로 쓰면 된다.

역할 전투 버티컬 슬라이스 + 측정

새로 짓지 않는다. 있는 조각을 "합산이 보이는 데모 + 독립 만료 + 측정"으로 완성.

한 줄 — 단일 러너·Effect 조합·`ApplyStatModifier`·`StatusRunner`·역할/표식은 이미 있다. 8월엔 **"버프(+공격력)와 방깅(-방어력)이 한 데미지 숫자로 합산되는 걸 눈에 보이게 + 각자 독립 만료 + GC 측정"**으로 대표작 #1을 **시연 가능한 데모**로 만든다.

왜 — 지금 아쉬운 3가지를 메움

아쉬움	이 계획이 메우는 법
공유 파이프라인으로 뭐가 벌어지는지 안 보임	버프·방깅·(표식)이 한 데미지에 합쳐지는 머니샷 데모
태그·상태가 아무 일도 안 함	방깅/버프가 실제 스탯에 걸리고 만료 + (선택)Marked 게이팅
성능 숫자 0	캐스트 GC/alloc Before/After 측정

머니샷 — 핵심 데모 장면

포인트는 이 공식이 화면/로그에 보이는 것. VFX 반짝임이 아니라 "여러 modifier가 정확히 합산·독립관리".

```
// 브레이커 → 적에게 [방어력 -] (+선택 Marked) ← 몹이 Effect를 받음
// 버퍼 → 아군에게 [공격력 +] ← 플레이어가 Effect를 받음
// 딜러 → 때림
```

최종 데미지 = 공격력(버프 +30%) vs 방어력(방깅 -40%) (+Marked 보너스)
 ↳ 한 공식이 여러 modifier를 읽고, 각 modifier는 소스별 독립 만료

무엇을 / 안 하나

✓ 한다

배선(자산 연결) · 데미지 공식 가시화(유일한 신규) · Before/After 데모 GIF · GC 측정

✗ 안 한다

Effect·캐릭터·스킬 수 늘리기(넓이=함정) · 가차/로스터 UI · 멀티 동기화 · 트라이포드 레이어(철회)

새로 "만드는" 건 사실상 데미지 공식 표시 하나. 나머지는 있는 걸 활성화·시연·측정.

4주 계획 — 주차별 산출물 · 검증

WEEK 1 활성화(배선) — 효과가 실제로 걸리게

브레이커=적에 방깅 `ApplyStatModifier(Target)` (bullet `_onHitEffects` 또는 E) · 버퍼=파티에 `+Atk(Party)` E · 각 캐릭 `RoleData` /E 자산 배선 점검.

검증: 적 Defense 내려갔다 duration 후 복구 / 파티 Attack 올랐다 복구(StatusRunner 만료). 산출: 3역할 한 파티 전투 진입, 효과가 스탯 반영.

WEEK 2 가시화(핵심·유일 신규) — 데미지 공식 표시

데미지 계산 시 항 노출: `base Atk · 버프 mod · target Def · 방깅 mod · (Marked)` → 최종 . 히트마다 디버그 오버레이/온스크린 로그.

검증: 최종 데미지=공식대로 · 항이 보임 · 버프만 만료돼도 방깅 유지(소스별 독립 제거). 산출: "공격력(+30%) vs 방어력(-40%) → XXX" 화면.

WEEK 3 데모화 — Before/After 슬라이스

짧은 전투: ①딜러 단독 ②표식→버프 없고 타격 → 데미지 점프. (선택 깊이) **Marked**가 딜러 추가데미지 게이팅(태그 실사용). GIF 2개 캡처.

검증: 콤보 시 데미지 눈에 띄게 점프, GIF 대비. 산출: Before/After GIF 2.

WEEK 4 측정 + 정리

캐스트 경로 GC/alloc Before/After(Profiler) — `CreateRuntime()⇒this` 무할당 증명. 포폴 1페이지=머니샷 GIF+정량지표(발동 3→1·새스킬 코드0·합산)+소스추적 서술.

검증: 캐스트 힙 할당 0(또는 급감) 스크린샷 · 1p 초안. 산출: 측정 스크린샷 + 포트폴리오 1페이지.

측정 계획 (약점 보강)

GC 측정

무엇: 어빌리티 1회 캐스트의 힙 할당(B). 도구: Unity Profiler(GC Alloc)/Recorder. 가설:

`CreateRuntime()⇒this` (stateless)+ctx 재사용이라 캐스트당 ≈ 0 . 주의: 에디터 측정은 과대계상 → 상대 (Before/After)로 제시, 절대값 단정 금지.

리스크 / 결정 포인트

- (a) 데미지가 이미 **target Defense**를 읽는가 — 읽으면(`max(dmg-def, 1)`) 방깅 자동 반영, 가시화만 추가. 안 읽으면 W1에 계산식 점검 먼저.
- (b) **Marked** 게이팅 포함 여부 — 코어(버프+방깅)는 태그 없이 성립. 시간 남으면 W3 +1.
- (c) 디버그 표시 방식 — 온스크린 텍스트 vs 기존 데미지 토스트 확장. 가장 빨리 되는 것.

성공 기준 한 줄

"버프와 방깅이 한 데미지 숫자로 합산되고 각자 독립 만료되는 걸, 데모 영상과 측정으로 보여준다."

